PROJETO G2 / MARCO 3 / ISSUES #16 E #18

Do formulário à API. Entenda cada passo.

Caderno de estudo · SvelteKit e TypeScript · 21/10/2026

01 / O que estamos construindo?

A issue #16 amplia o módulo da API: além de ler dados, ele passa a criar e atualizar tarefas e projetos com funções TypeScript. Essas funções podem ser chamadas pelas telas do projeto.

A issue #18 pede uma página em `/tarefas/nova`. Nela, a pessoa preenche os dados, salva a tarefa na API e volta para a lista de tarefas.

Também há um atalho: se a pessoa começar dentro de um projeto, esse projeto deve aparecer selecionado no formulário.

1. A pessoa abre **Nova tarefa**.
2. A página busca os projetos e mostra o formulário.
3. A pessoa preenche e envia os dados.
4. A API grava a tarefa e gera o ID.
5. O site abre a lista de tarefas novamente.

Como as duas entregas se conectam: a #16 fornece as quatro funções de escrita; a #18 usa `criarTarefa` na tela de cadastro. As funções para atualizar e para criar projetos ficam disponíveis para as próximas telas. O formulário reutilizável pertence à #17 e a validação completa à #20; essas duas issues continuam separadas.

02 / Onde encontrar o código

No VS Code, pressione **CmdP**, digite um dos caminhos abaixo e pressione Enter. O nome `+page.svelte` aparece várias vezes porque cada pasta representa uma página diferente.

Arquivo, a partir da raiz do projeto	Responsabilidade
`src/routes/tarefas/nova/+page.svelte`	Mostra o cadastro, guarda os valores preenchidos e coordena o envio.
`src/lib/servicos/api.ts`	Conversa com a API. Reúne as funções de leitura e as quatro funções de criação e atualização.
`src/tipos.ts`	Mantém o contrato de `Tarefa` e `Projeto` e define os tipos derivados usados para criar e atualizar.
`src/routes/+page.svelte`	É a lista, no endereço `/`. Tem a entrada para criar uma tarefa.
`src/routes/projetos/[id]/+page.svelte`	É o detalhe de um projeto. Seu link leva o ID do projeto até o cadastro.
`tests/api.test.mjs`	Verifica o comportamento das funções da API, como método, endereço, corpo JSON e tratamento de erros.

Um arquivo Svelte costuma juntar três partes:

- `<script lang="ts">`: TypeScript, com dados e funções.

- Marcação HTML/Svelte: os campos, textos e botões que aparecem.

- `<style>`: CSS, que organiza a aparência da página.

Os exemplos deste guia são recortes para estudar os conceitos; os arquivos do projeto são a referência para o código completo.

03 / Como o endereço encontra a página?

```
src/routes/tarefas/nova/+page.svelte
                    v
               /tarefas/nova

src/routes/tarefas/[id]/+page.svelte
                    v
          /tarefas/2, /tarefas/abc...
```

`nova` é uma parte fixa do endereço. `[id]` é uma parte variável. O SvelteKit escolhe a rota fixa antes da variável; por isso, abrir `/tarefas/nova` deve mostrar o cadastro, sem tentar buscar uma tarefa cujo ID seja “nova”.

E o projeto já selecionado?

O link do projeto leva uma informação extra depois do ponto de interrogação:

```
/tarefas/nova?projetoId=3
```

`/tarefas/nova` escolhe a página. `projetoId=3` sugere qual projeto selecionar. Isso é um **parâmetro de consulta**, também chamado de *query parameter*.

```
import { page } from '$app/state';

page.url.searchParams.get('projetoId'); // "3"
```

`page` vem do SvelteKit e informa a URL atual. Ele não é o roteador. Também não confundimos a consulta acima com `page.params.id`, que lê a parte `[id]` de uma rota dinâmica.

Antes de selecionar, o código confere se esse ID existe nos projetos que vieram da API. Se alguém escrever um ID inválido na URL, o formulário deve orientar a pessoa a escolher um projeto válido.

Por que usar `$derived` e `$effect` aqui?

O SvelteKit pode manter a mesma página aberta quando só a consulta muda. Por isso, observar a sugestão da URL permite atualizar o projeto ao ir de `?projetoId=1` para `?projetoId=2`. O efeito observa a sugestão e os projetos carregados; ele não deve observar o campo que a pessoa edita, para não desfazer a escolha manual dela.

04 / Como o formulário guarda o que eu digito?

Os dados enviados são: `projetoId`, `titulo`, `descricao`, `status`, `prioridade`, `responsavel` e `prazo`. Os nomes precisam respeitar o contrato que a API usa.

`NovaTarefa`: o formato dos dados

```
export type NovaTarefa = Omit<Tarefa, 'id'>;
```

`Omit` significa “usar os campos de `Tarefa`, retirando `id`”. O usuário não inventa o ID. O `json-server` o gera quando grava a tarefa.

O TypeScript ajuda a detectar nomes e tipos errados enquanto programamos. Ele não substitui as verificações do formulário nem a validação de um servidor real.

`$state`: um valor que atualiza a tela

```
let titulo = $state('');
```

Esse recorte começa com um título vazio. Quando o estado muda, o Svelte atualiza a parte da tela que depende dele. Na página completa, os campos podem estar agrupados em um objeto.

`bind:value`: ligar o campo ao estado

```
<label for="titulo">Título</label>
<input id="titulo" bind:value={titulo} required />
```

Ao digitar no campo, `titulo` recebe o valor. Se o código mudar `titulo`, o campo também muda. O `label` dá nome ao campo; o `for` aponta para seu `id`. `required` pede ao navegador que não envie esse campo vazio.

`onMount`: buscar as opções depois de abrir

`listarProjetos()` fornece as opções do seletor. `onMount` executa a busca quando a página é montada no navegador. Enquanto a resposta não chega, mostramos carregamento; se falhar, mostramos uma mensagem e permitimos tentar de novo.

Duas ideias diferentes: `onMount` inicia essa busca. O efeito acompanha a sugestão de projeto da URL. Um não substitui automaticamente o outro.

05 / O que acontece ao clicar em Salvar?

Um formulário trata seu envio com uma função. Ela impede o envio tradicional do navegador, prepara os dados e aguarda a API.

Operação	O que ela faz no projeto
`GET /projetos`	Lê os projetos para o seletor.
`POST /tarefas`	Envia os dados para criar uma tarefa.
`goto('/')`	Abre a página da lista depois da confirmação de criação.

Recorte do formato de uma requisição de criação:

```
fetch('http://localhost:3000/tarefas', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(dados)
});
```

- `method: 'POST'` informa que queremos criar um registro.

- `Content-Type` informa que o corpo usa JSON.

- `JSON.stringify(dados)` transforma o objeto TypeScript/JavaScript em texto JSON para enviar.

- `criarTarefa(dados)` reúne essa conversa com a API em uma função que a página chama.

Por que `async` e `await`?

```
const tarefaCriada = await criarTarefa(dados);
await goto('/');
```

A API leva algum tempo para responder. `await` espera o resultado dentro de uma função `async`. Só depois de criar a tarefa a navegação é iniciada.

A lista fica em `/`, e não em `/tarefas`. Ao voltar para ela, a busca da lista carrega os registros, incluindo a tarefa nova.

E se der erro?

`try` tenta a operação e `catch` trata uma falha. Se a gravação falhar, os campos continuam preenchidos para a pessoa tentar novamente. Durante o envio, o botão fica desabilitado para evitar pedidos repetidos.

Se a API já confirmou a criação, uma falha ao navegar não deve habilitar uma segunda gravação da mesma tarefa. A página reconhece o sucesso e permite voltar à lista. O `onDestroy` também marca quando você saiu desta tela, para uma resposta atrasada não forçar a navegação depois disso.

06 / A API agora também escreve

Antes, o módulo da API sabia ler: listar tarefas, buscar uma tarefa e carregar projetos. A #16 acrescenta quatro funções. A página passa os dados para elas; cada função monta a requisição correta e devolve a resposta da API.

Função e retorno	Pedido enviado
`criarTarefa(dados: NovaTarefa): Promise<Tarefa>`	`POST /tarefas`. Envia os campos da nova tarefa e recebe a tarefa criada, com ID.
`atualizarTarefa(id: string, dados: AtualizacaoTarefa): Promise<Tarefa>`	`PATCH /tarefas/ID`. Envia os campos que devem mudar e recebe a tarefa atualizada.
`criarProjeto(dados: NovoProjeto): Promise<Projeto>`	`POST /projetos`. Envia os campos do novo projeto e recebe o projeto criado, com ID.
`atualizarProjeto(id: string, dados: AtualizacaoProjeto): Promise<Projeto>`	`PATCH /projetos/ID`. Envia os campos que devem mudar e recebe o projeto atualizado.

Essas assinaturas são um mapa: o que está entre parênteses é a entrada; o que vem depois de `:` é o tipo do retorno.

`Omit`, `Partial` e `Promise<T>`

```
export type NovaTarefa = Omit<Tarefa, 'id'>;
export type AtualizacaoTarefa = Partial<NovaTarefa>;

export type NovoProjeto = Omit<Projeto, 'id'>;
export type AtualizacaoProjeto = Partial<NovoProjeto>;
```

- `Omit` retira um campo de um tipo. Na criação, retiramos `id` porque a API o gera.

- `Partial` torna os campos opcionais. Podemos atualizar só `status`, sem repetir o restante da tarefa.

- `Promise<Tarefa>` representa um resultado futuro. Depois do `await`, obtemos uma `Tarefa` se o pedido tiver sucesso. Uma falha precisa ser tratada.

São tipos derivados: os campos e os nomes do contrato original de `Tarefa` e `Projeto` continuam os mesmos. A tipagem auxilia o desenvolvimento, mas não valida sozinha os dados recebidos pela rede.

Quando usar GET, POST, PATCH ou PUT?

Método	Ideia e uso aqui
`GET`	Ler. Exemplo: carregar os projetos do seletor.
`POST`	Criar. Aqui, enviamos um novo registro à coleção `/tarefas` ou `/projetos`.
`PATCH`	Alterar parte de um registro existente. No json-server deste projeto, enviamos um objeto JSON com os campos que devem mudar.
`PUT`	Substituir a representação completa de um registro. Não é o método escolhido por estas funções, que aceitam atualizações parciais.

O formato aceito por um PATCH depende da API. Aqui usamos o contrato do json-server; isso não significa que todo servidor aceite o mesmo corpo JSON. A distinção entre alteração parcial e substituição é descrita na especificação de PATCH.

Atualizar uma tarefa: ID na URL, mudanças no corpo

```
const tarefa = await atualizarTarefa('2', {
  status: 'em-andamento'
});
```

O `'2'` identifica qual tarefa queremos alterar. O objeto informa o que deve mudar. O pedido corresponde a `PATCH /tarefas/2` com o corpo `{ "status": "em-andamento" }`. Os demais campos são preservados pelo json-server.

A função usa `encodeURIComponent(id)` ao montar o endereço. Assim, caracteres especiais do ID não viram separadores de caminho ou de consulta da URL por acidente. O ID é recebido separadamente e não é um campo de `AtualizacaoTarefa`.

Criar e depois atualizar um projeto

```
const projeto = await criarProjeto({
  nome: 'Estudo de Svelte',
  descricao: 'Exercícios da equipe',
  disciplina: 'Desenvolvimento Front-end',
  criadoEm: '2026-10-21'
});

const atualizado = await atualizarProjeto(projeto.id, {
  descricao: 'Exercícios e revisão da equipe'
});
```

Quem chama `criarProjeto` fornece `criadoEm`, junto com os outros campos exigidos. A função de serviço não inventa essa data; a API gera somente o ID. A atualização do exemplo altera apenas a descrição.

Por que existe o auxiliar `requisitar`?

As funções precisam fazer tarefas em comum: chamar `fetch`, verificar a resposta e ler o JSON. O auxiliar centraliza esses passos para não repeti-los em cada operação.

```
async function requisitar<T>(
  caminho: string,
  opcoes?: RequestInit
): Promise<T>
```

`RequestInit` é o tipo das opções aceitas pelo `fetch`, como `method`, `headers` e `body`. O `?` torna esse argumento opcional. As leituras podem chamar apenas com o caminho; as escritas passam suas opções.

```
return requisitar<Tarefa>(
  `/tarefas/${encodeURIComponent(id)}`,
  {
    method: 'PATCH',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(dados)
  }
);
```

O `T` indica o tipo de resposta esperado por quem chama. Em `requisitar<Tarefa>`, esperamos uma tarefa; em `requisitar<Projeto>`, esperamos um projeto.

Como `ErroApi` ajuda?

Se a resposta HTTP não indicar sucesso, o auxiliar lança `ErroApi`, guardando o código em `status`. Uma tela pode distinguir, por exemplo, um registro não encontrado (`404`) de outra falha.

```
try {
  await atualizarTarefa('2', { prioridade: 'alta' });
} catch (erro) {
  if (erro instanceof ErroApi && erro.status === 404) {
    // Mostrar que a tarefa não foi encontrada.
  } else {
    // Mostrar que não foi possível salvar a alteração.
  }
}
```

Uma falha de conexão pode ocorrer antes de existir resposta HTTP. Nesse caso, o erro original do `fetch` também chega ao `catch`; não presumimos que todo erro seja um `ErroApi`.

Como chegar a esta parte no código: abra `src/lib/servicos/api.ts` e procure as quatro funções da tabela. Depois abra `src/tipos.ts` para ver seus tipos de entrada. Na tela de nova tarefa, procure a chamada de `criarTarefa`: ela é a ligação entre as issues #18 e #16.

07 / Como abrir pelo terminal

Esta entrega usa uma cópia de trabalho separada. A raiz dessa cópia é:

```
/Users/andreybuenoisoton/Documents/Codex/2026-10-14/create-a-new-site-with-sites-4/outputs/marco3-issue18
```

Abra essa pasta no VS Code. No terminal integrado, use `pwd` para confirmar onde você está. Todos os comandos abaixo partem da raiz do projeto, onde está `package.json`.

1. Instalar as dependências, quando necessário

```
npm ci
```

Instala as versões registradas no `package-lock.json`. É necessário em uma cópia recém-baixada ou quando as dependências precisam ser reinstaladas. O projeto pede Node.js 24 ou superior; `node --version` mostra sua versão.

2. Abrir a API em uma aba do terminal

```
npm run api
```

Inicia o `json-server` usando o `db.json` dessa pasta. O endereço esperado pela aplicação é `http://localhost:3000`. Esse terminal continua ocupado atendendo à API.

3. Abrir o site em outra aba do terminal

```
npm run dev
```

Inicia o servidor de desenvolvimento. Abra o endereço **Local** que ele imprimir, normalmente `http://localhost:5173`, e acrescente `/tarefas/nova`.

Não copie o prompt. Um trecho como `andrey@Mac projeto-g2-frontend %` apenas mostra o usuário e a pasta atual. Você digita somente o comando depois do `%`.

Com um servidor rodando, aquela aba não está pronta para comandos Git. Abra outra aba. Para encerrar um servidor que você iniciou, use **Ctrl+C** na aba dele.

O site e a API são dois programas. A porta **5173** mostra a interface; a porta **3000** responde com os dados. Abrir a API diretamente no navegador pode mostrar JSON, o que é esperado.

Conferir os tipos e preparar a aplicação

```
npm run check
npm run build
```

O primeiro procura problemas de tipos e de Svelte. O segundo gera a versão de produção. Eles ajudam a conferir o código, mas não substituem experimentar o formulário no navegador.

Executar os testes das funções da API

```
node --test tests/api.test.mjs tests/api.integration.test.mjs
npx tsc -p tests/tsconfig.json
```

Execute na raiz do projeto. O primeiro comando usa o executor de testes do Node: confere os pedidos montados e faz operações HTTP contra um json-server com banco em memória, sem tocar no db.json da equipe. O segundo executa o TypeScript para conferir entradas e retornos: ele verifica também que id e campos inválidos sejam recusados. Leia o resultado impresso no terminal.

08 / O que você deve experimentar

Use esta lista como roteiro de estudo. As caixas são suas marcações locais e não representam um relatório automático de testes.

- Abrir `/tarefas/nova` e identificar a tela de cadastro.

- Abrir um projeto e usar seu link de nova tarefa. Conferir qual opção já vem selecionada.

- Trocar o projeto manualmente e perceber que sua escolha é mantida.

- Preencher todos os campos e salvar uma tarefa de teste. Conferir se ela aparece na lista.

- Observar o botão durante o envio e explicar por que ele fica desabilitado.

- Abrir um detalhe como `/tarefas/2` e perceber que continua sendo outra página.

Verificação da tela e da API

Na verificação inicial da #18, o Svelte encontrou zero erros e avisos e a compilação passou. No navegador, conferimos cadastro, projeto pré-selecionado, troca manual, campos obrigatórios, clique duplo e recuperação após falha de conexão, usando uma cópia do banco.

Na entrega da #16, passaram 18 testes automatizados, a verificação dos contratos TypeScript, a checagem do Svelte sem erros nem avisos e a compilação. Foram conferidos POST e PATCH, ID no caminho, JSON no corpo, preservação de campos, erros HTTP e leituras existentes. Os testes de integração usaram banco em memória e encerraram seu próprio servidor. Nenhuma tela foi modificada nesta etapa da #16.

Criar ou atualizar registros grava dados na API em uso. Para estudar sem alterar os dados da equipe, faça os testes em uma cópia do banco.

09 / Agora explique com suas palavras

Tente responder antes de abrir a resposta. Se travar, volte ao trecho do guia e procure a ligação entre a URL, o campo e a API.

1. Por que /tarefas/nova não abre o detalhe de uma tarefa chamada “nova”?

Porque existe uma rota fixa em `tarefas/nova/+page.svelte`. O SvelteKit dá prioridade à rota fixa sobre `tarefas/[id]/+page.svelte`.

2. De onde vem o projeto que aparece selecionado?

Do parâmetro `projetoId` na consulta da URL, enviado pelo link da página do projeto. O código lê essa sugestão e confere se ela corresponde a um projeto existente nas opções da API.

3. Qual é a diferença entre GET e POST nesta tela?

`GET` lê os projetos. `POST` envia os dados para criar a tarefa. A requisição de criação leva um corpo JSON.

4. Por que não enviamos um id junto com os campos?

O ID é gerado pela API. O tipo `NovaTarefa` usa os campos de `Tarefa` sem `id`, deixando isso claro enquanto programamos.

5. O que aconteceria se voltássemos para a lista antes de esperar a API?

A pessoa poderia ver a lista antes da gravação terminar ou sair do formulário sem perceber uma falha. A ordem correta é confirmar a criação e depois navegar.

6. O que bind:value faz quando você digita um título?

Atualiza o estado ligado ao campo. A ligação também funciona no outro sentido: quando o código altera esse estado, o valor mostrado no campo acompanha a mudança.

7. Para mudar só a prioridade, uso criarTarefa ou atualizarTarefa?

Uso `atualizarTarefa`, passando o ID do registro e um objeto como `{ prioridade: 'alta' }`. O pedido usa PATCH. `criarTarefa` usa POST para acrescentar uma nova tarefa.

8. Qual é a diferença entre Omit e Partial nesses tipos?

`Omit<Tarefa, 'id'>` retira o ID do tipo de criação. `Partial<NovaTarefa>` torna opcionais os campos da atualização, permitindo enviar apenas aqueles que devem mudar.

9. No cadastro de um projeto, quem fornece criadoEm?

O código que chama `criarProjeto` fornece esse campo no objeto `NovoProjeto`. A função envia os dados recebidos; a API gera o ID, sem inventar a data de criação.

10. Onde vão o ID e os novos valores em uma atualização?

O ID identifica o registro na URL, como `/tarefas/2`. Os valores que devem mudar vão no corpo JSON do PATCH. A função codifica o ID com `encodeURIComponent` ao montar o endereço.

11. Promise<Projeto> quer dizer que já tenho um projeto pronto?

Não. É um resultado que pode chegar depois. Ao usar `await`, esperamos a operação; se ela tiver sucesso, recebemos o projeto. Se falhar, tratamos o erro no `catch`.

12. Por que todas as funções usam requisitar?

Porque a chamada de rede, a leitura do JSON e a checagem de sucesso são passos comuns. O auxiliar recebe o caminho e opções `RequestInit`, evitando repetir esses passos em cada função.

Para consultar quando precisar

- SvelteKit: páginas e rotas
- SvelteKit: prioridade entre rotas
- SvelteKit: informações da página atual
- SvelteKit: navegar com goto
- SvelteKit: preservação da página ao navegar
- Svelte: estado com $state
- Svelte: ligação com bind
- TypeScript: Partial
- TypeScript: Omit
- HTTP: especificação do método PATCH